



National Technical University of Athens

School of Electrical and Computer Engineering

8th Semester

VLSI Digital Systems

Laboratory Exercise 1

Dimitris Vatalas

(Student ID: 03121093)

Aristotelis Georgios Dritsas

(Student ID: 03122127)

Athens, March 2026

Contents

Exercise A.2: 3-to-8 Binary Decoder	2
Exercise B.2: 4-bit Shift Register with Parallel Loading	8
Exercise B.3: 3-bit Counter with Enable Input and Output Carry	11
1. Up/Down Counter without Limit	11
2. Up Counter with Limit	14
Exercise 1: Half Adder using Dataflow Description	17
Exercise 2: Full Adder using Structural Description	20
Exercise 3: 4-bit Parallel Adder using Structural Description	23
Exercise 3: BCD Full Adder using Structural Description	26
Exercise 4: 4-Digit BCD Parallel Adder using Structural Description	30

Exercise A.2: 3-to-8 Binary Decoder

The VHDL code used to describe the decoder using a dataflow architecture is presented below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

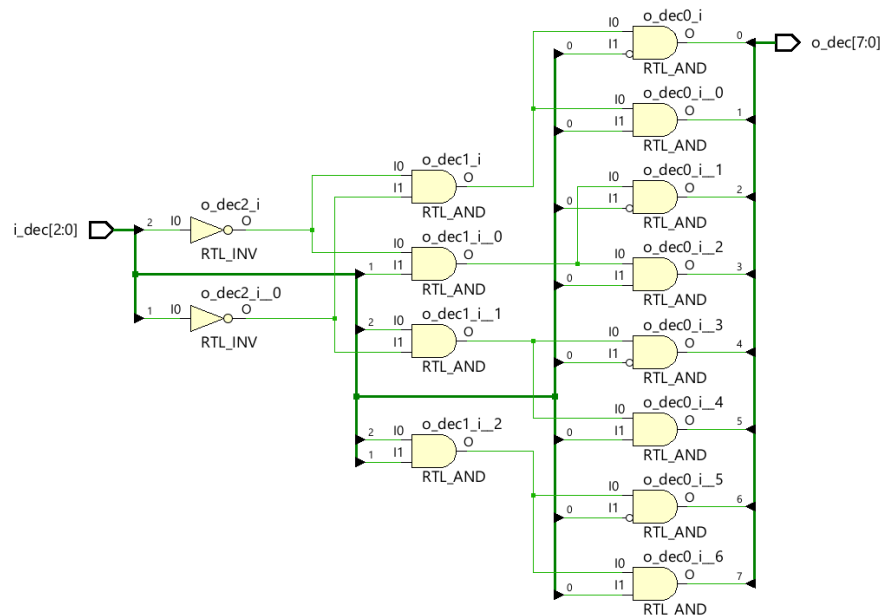
entity bin_dec is
    port
    (
        i_dec : in std_logic_vector(2 downto 0);
        o_dec : out std_logic_vector(7 downto 0)
    );
end bin_dec;

architecture Dataflow of bin_dec is

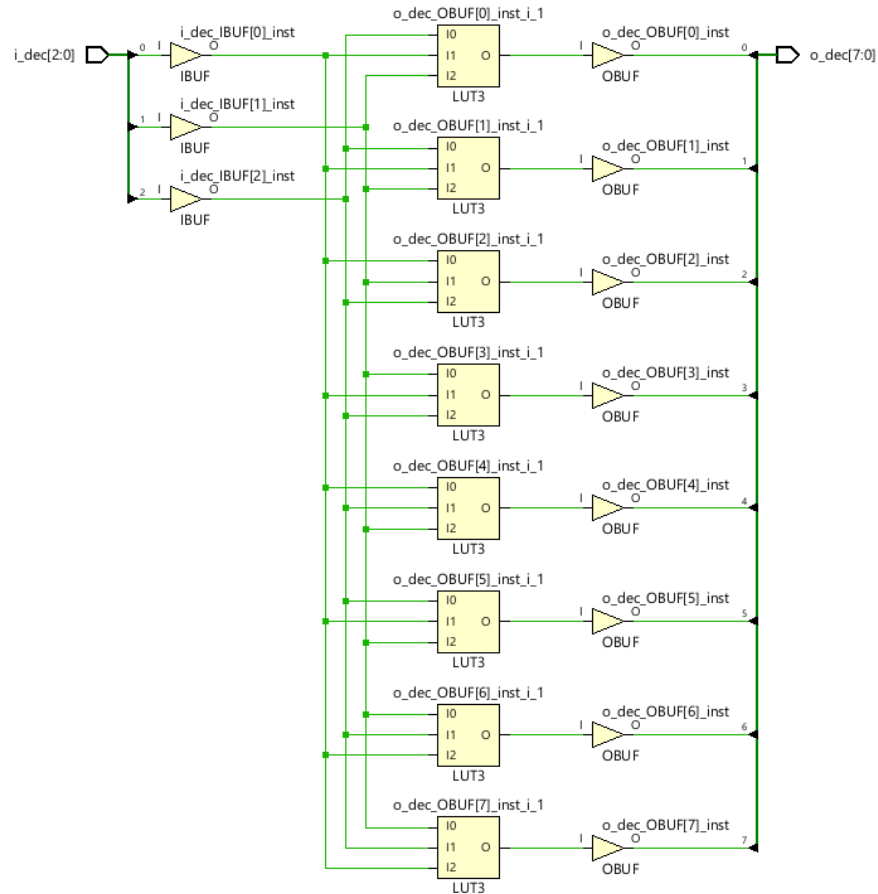
begin
    o_dec(0) <= (not i_dec(2)) and (not i_dec(1)) and (not i_dec(0));
    o_dec(1) <= (not i_dec(2)) and (not i_dec(1)) and i_dec(0);
    o_dec(2) <= (not i_dec(2)) and i_dec(1) and (not i_dec(0));
    o_dec(3) <= (not i_dec(2)) and i_dec(1) and i_dec(0);
    o_dec(4) <= i_dec(2) and (not i_dec(1)) and (not i_dec(0));
    o_dec(5) <= i_dec(2) and (not i_dec(1)) and i_dec(0);
    o_dec(6) <= i_dec(2) and i_dec(1) and (not i_dec(0));
    o_dec(7) <= i_dec(2) and i_dec(1) and i_dec(0);

end Dataflow;
```

The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



In order to verify the correct operation of the decoder, all possible input combinations are generated and the corresponding outputs are observed:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity bin_dec_tb is
-- Port ( );
end bin_dec_tb;

architecture tb of bin_dec_tb is
-- COMPONENT
component bin_dec is
port
(
i_dec : in std_logic_vector(2 downto 0);
o_dec : out std_logic_vector(7 downto 0)
);
end component;

-- SIGNALS
signal i_dec      : std_logic_vector(3-1 downto 0) := (others => '0');
signal o_dec      : std_logic_vector(8-1 downto 0) := (others => '0');

-- CONSTANTS

```

```

    constant TIME_DELAY : time := 10 ns;

begin

    DUT : bin_dec
        port map (
            i_dec    => i_dec,
            o_dec    => o_dec
        );

    STIMULUS : process
    begin
        i_dec <= "000";
        wait for (1 * TIME_DELAY);

        i_dec <= "001";
        wait for (1 * TIME_DELAY);

        i_dec <= "010";
        wait for (1 * TIME_DELAY);

        i_dec <= "011";
        wait for (1 * TIME_DELAY);

        i_dec <= "100";
        wait for (1 * TIME_DELAY);

        i_dec <= "101";
        wait for (1 * TIME_DELAY);

        i_dec <= "110";
        wait for (1 * TIME_DELAY);

        i_dec <= "111";
        wait for (1 * TIME_DELAY);

        wait;

    end process;
end tb;

```

The simulation results are shown below:



The correct correspondence between each input combination and its respective output can be observed. The VHDL code used to describe the decoder using a behavioral architecture is presented below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity bin_dec is
    port
    (
        i_dec : in std_logic_vector(2 downto 0);
        o_dec : out std_logic_vector(7 downto 0)
    );
end bin_dec;

architecture Behavioral of bin_dec is

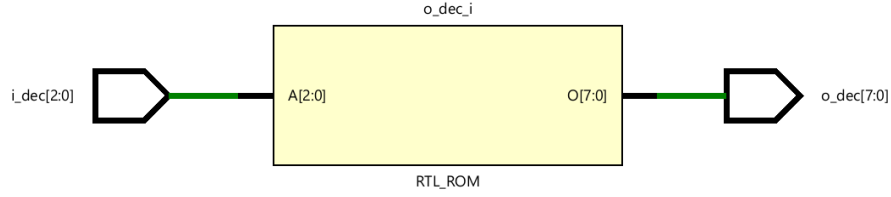
begin

    ENC_MODULE: process(i_dec)
    begin
        case i_dec is
            when "000" => o_dec <= "00000001";
            when "001" => o_dec <= "00000010";
            when "010" => o_dec <= "00000100";
            when "011" => o_dec <= "00001000";
            when "100" => o_dec <= "00010000";
            when "101" => o_dec <= "00100000";
            when "110" => o_dec <= "01000000";
            when "111" => o_dec <= "10000000";
            when others => o_dec <= "00000000";
        end case;
    end process;

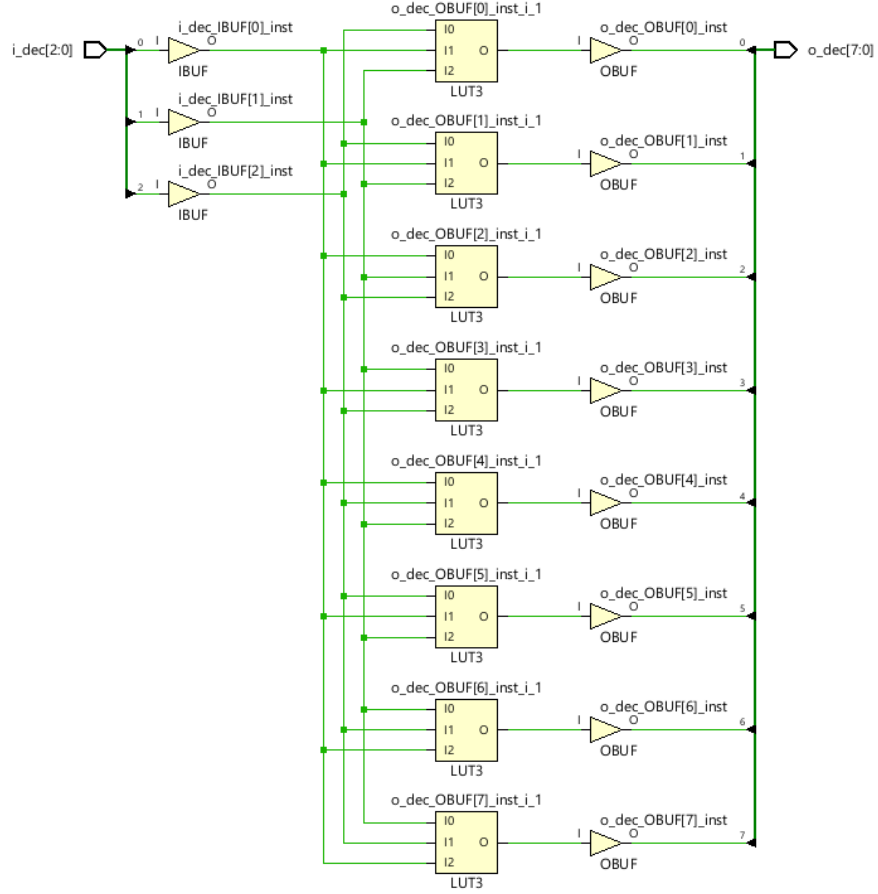
end Behavioral;

```

The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



It can be observed that the schematics produced by the Elaborate Design stage are different for the two architectures, since they depend directly on the way the architecture has been described in the source code. In contrast, the schematic obtained after synthesis is identical in both cases. This demonstrates that, although the decoder is described using two different coding styles, synthesis results in the same final logic structure and, consequently, the same FPGA implementation.

In order to verify the correct operation of the decoder, all possible input combinations are generated and the corresponding outputs are observed:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity bin_dec_tb is
-- Port ( );
```

```

end bin_dec_tb;

architecture tb of bin_dec_tb is
    -- COMPONENT
    component bin_dec is
        port
        (
            i_dec : in std_logic_vector(2 downto 0);
            o_dec : out std_logic_vector(7 downto 0)
        );
    end component;

    -- SIGNALS
    signal i_dec      : std_logic_vector(3-1 downto 0) := (others => '0');
    signal o_dec      : std_logic_vector(8-1 downto 0) := (others => '0');

    -- CONSTANTS
    constant TIME_DELAY : time := 10 ns;

begin

    DUT : bin_dec
        port map (
            i_dec      => i_dec,
            o_dec      => o_dec
        );

    STIMULUS : process
    begin
        i_dec <= "000";
        wait for (1 * TIME_DELAY);

        i_dec <= "001";
        wait for (1 * TIME_DELAY);

        i_dec <= "010";
        wait for (1 * TIME_DELAY);

        i_dec <= "011";
        wait for (1 * TIME_DELAY);

        i_dec <= "100";
        wait for (1 * TIME_DELAY);

        i_dec <= "101";
        wait for (1 * TIME_DELAY);

        i_dec <= "110";
        wait for (1 * TIME_DELAY);

        i_dec <= "111";
        wait for (1 * TIME_DELAY);

        wait;
    end process;
end tb;

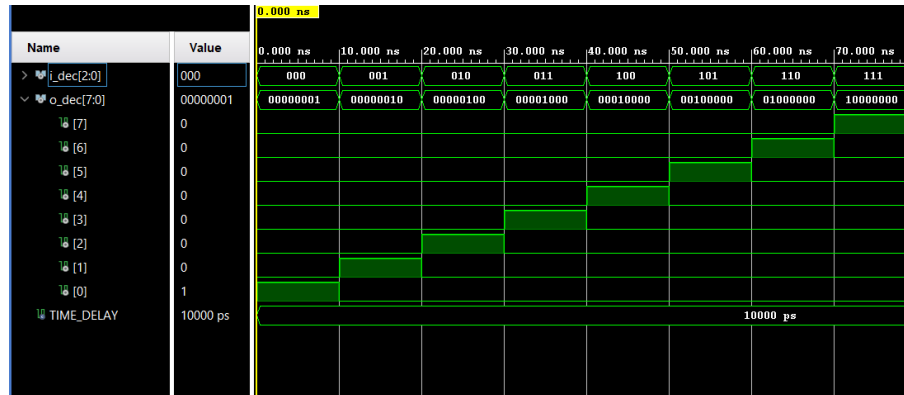
```

```

    end process;
end tb;

```

The simulation results are shown below:



Once again, the correct correspondence between each input combination and its respective output can be observed.

Exercise B.2: 4-bit Shift Register with Parallel Loading

The VHDL code used to implement the shift register is presented below. The register includes an additional `std_logic` input used to select between left and right shifting.

It is worth noting that the code also includes an additional output, `ins_test`, which is not necessary for the actual operation of the circuit. It is included to facilitate simulation and verification by allowing the value currently stored in the shift register to be directly observed at any given time.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg is
    Port (
        clk,rst,si,en,pl,ch_way: in std_logic;
        din: in std_logic_vector(3 downto 0);
        so: out std_logic;
        ins_test :out std_logic_vector(3 downto 0)
    );
end shift_reg;

architecture rtl of shift_reg is

    signal dff: std_logic_vector(3 downto 0);
    signal sf: std_logic;
begin
    edge: process (clk,rst)
    begin
        if rst='0' then --reset is negative logic
            dff<=(others=>'0');
            sf <= '0';
        elsif clk'event and clk='1' then
            if pl='1' then
                dff<=din;
            end if;
        end if;
    end process;
end rtl;

```

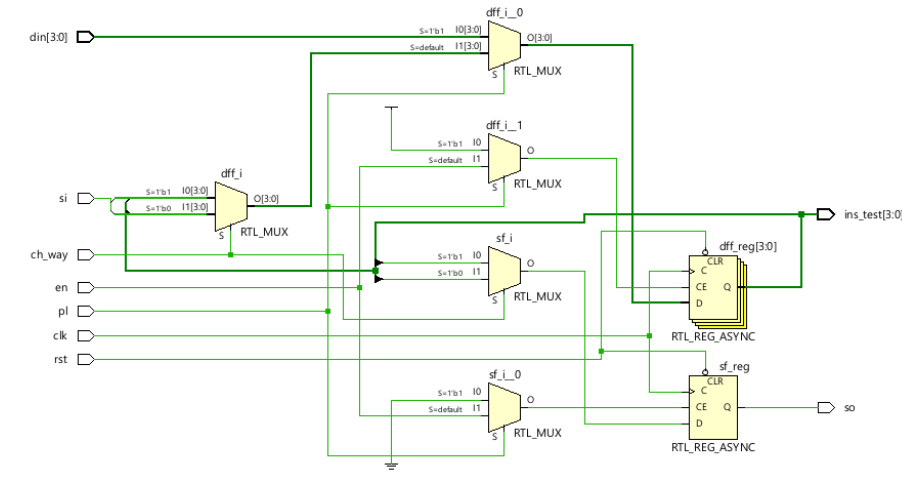
```

    elsif en='1' then
        case ch_way is
            when '1'=>
                sf <= dff(0);
                dff<=si&dff(3 downto 1);
            when '0'=>
                sf <= dff(3);
                dff<=dff(2 downto 0)&si;
            when others =>
                null;
            end case;
        end if;
    end if;
end process;
so<=sf;
ins_test<= dff;

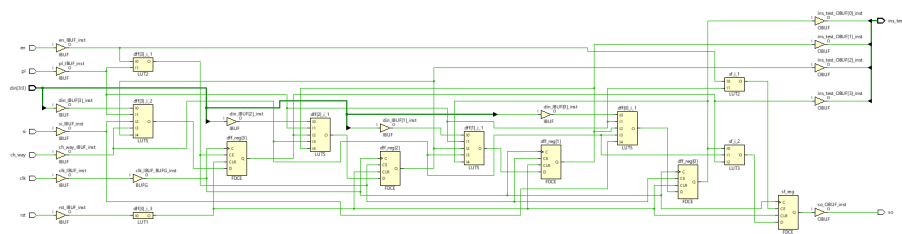
end rtl;

```

The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



The following testbench was implemented for the simulation:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg_tb is
    -- Port ( );
end shift_reg_tb;

```

```

architecture tb of shift_reg_tb is
    -- COMPONENT
    component shift_reg is
        Port (
            clk,rst,si,en,pl,ch_way: in std_logic;
            din: in std_logic_vector(3 downto 0);
            so: out std_logic;
            ins_test :out std_logic_vector(3 downto 0)
        );
    end component;
    -- SIGNALS
    signal clk,rst,si,en,pl,ch_way : std_logic ;
    signal din : std_logic_vector(3 downto 0);
    signal so: std_logic ;
    signal ins_test : std_logic_vector(3 downto 0) ;
    -- CONSTANTS
    constant CLOCK_PERIOD : time := 10 ns;

```

```

begin

```

```

    DUT : shift_reg
        port map (
            clk => clk,
            rst => rst,
            si  => si,
            en  => en,
            pl  => pl,
            ch_way => ch_way,
            din  => din,
            so   => so,
            ins_test =>ins_test
        );
    STIMULUS : process
    begin
        -----initialize signals-----
        rst    <= '0';    -- active low reset
        en     <= '0';
        pl     <= '0';
        ch_way <= '1';
        din    <= "1010";
        wait for CLOCK_PERIOD;

        -----load initial number-----
        rst <= '1';      -- release reset
        pl <= '1';
        wait for 4*CLOCK_PERIOD;

        -----test shift right-----
        pl <= '0';
        en <= '1';
        wait for 8*CLOCK_PERIOD;

        -----test reset-----
        rst <= '0';
        wait for 4*CLOCK_PERIOD;

        -----load new number-----
        en <= '0';
        rst <= '1';
        din <= "1001";

```

```

    pl  <= '1';
    wait for 4*CLOCK_PERIOD;

    -----test shift left-----
    pl  <= '0';
    en  <= '1';
    ch_way <= '0';
    wait for 8*CLOCK_PERIOD;

    -----test enable-----
    en <= '0';
    wait for 4*CLOCK_PERIOD;

    wait;
end process;

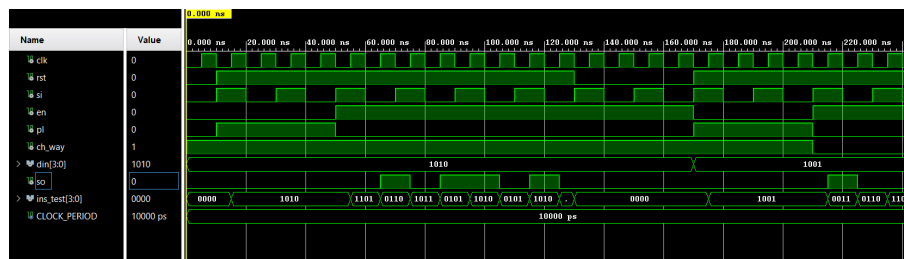
GEN_SI : process
begin
    loop
        si <= '0';
        wait for CLOCK_PERIOD;
        si <= '1';
        wait for CLOCK_PERIOD;
    end loop;
end process;

GEN_CLK : process
begin
    clk <= '0';
    wait for (CLOCK_PERIOD / 2);
    clk <= '1';
    wait for (CLOCK_PERIOD / 2);
end process;

end tb;

```

To verify the correct operation of the shift register, the main operating states of the circuit were examined, including reset, enable/disable, parallel data loading, and shifting in both directions. Since considering all possible parallel-load input combinations would result in a very large number of test cases, exhaustive verification was not practical. Therefore, representative test cases were selected to cover the main operating modes of the circuit.



It can be observed that the output values and signal waveforms confirm the correct operation of the implemented shift register.

Exercise B.3: 3-bit Counter with Enable Input and Output Carry

1. Up/Down Counter without Limit

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity counter_up_down is
    port(
        clk,resetn,count_en,up_down : in std_logic;
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end counter_up_down;

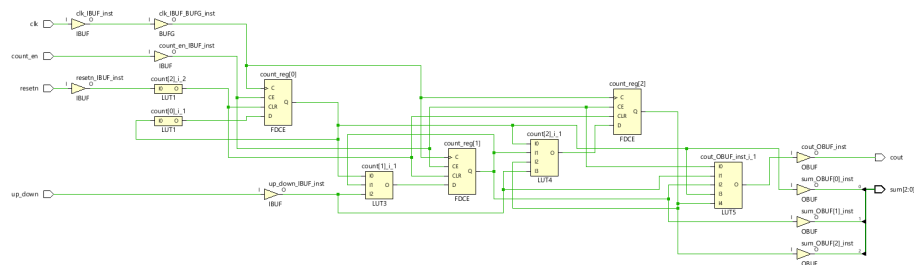
architecture rtl_nolimit of counter_up_down is

signal count: std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn='0' then --reset is negative logic
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en='1' then --enable counting positive logic
                case up_down is
                    when '1'=>
                        count<=count+1;
                    when '0' =>
                        count<=count-1;
                    when others =>
                        null;
                    end case;
                end if;
            end if;
        end process;
        sum <= count;
        cout <= '1' when (count = 7 and count_en = '1' and up_down = '1') or
            (count = 0 and count_en = '1' and up_down = '0')
            else '0';
    end rtl_nolimit;
end

```

The RTL schematic illustrates the internal logic of the counter module. It features a 3-bit counter implemented using an RTL_MUX (multiplexer) and an RTL_REG_ASYNC (asynchronous register). The counter's current value is stored in the register, which is also used to calculate the next value. The next value is determined by the current value plus or minus one, depending on the up_down input. The RTL_MUX selects between the current value and the next value based on the count_en input. The RTL_REG_ASYNC is clocked by clk and reset by resetn. The output of the counter is sum[2:0].

The following schematic is obtained after synthesis:



The following testbench was implemented for the simulation:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity count_up_down_nolim_tb is
-- Port ( );
end count_up_down_nolim_tb;

architecture tb of count_up_down_nolim_tb is

-- COMPONENT
component counter_up_down is
    port(
        clk, resetn, count_en, up_down : in std_logic;
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end component;

-- SIGNALS
signal clk, resetn, count_en, up_down : std_logic;
signal sum : std_logic_vector(2 downto 0);
signal cout : std_logic;

-- CONSTANTS
constant CLOCK_PERIOD : time := 10 ns;

begin
    DUT : counter_up_down
        port map(
            clk      => clk,
            resetn   => resetn,
            count_en => count_en,
            up_down  => up_down,
            sum      => sum,
            cout     => cout
        );

    STIMULUS : process
    begin

        up_down  <= '1';
        count_en <= '0';
        resetn  <= '0'; --initialize the counter to zero
        wait for (1*CLOCK_PERIOD);
```

```

resetn <='1';
count_en<='1'; --enable counting
wait for (14*CLOCK_PERIOD); --count two times from zero to 7

resetn <='0'; --test reset
wait for (1*CLOCK_PERIOD);

resetn <='1';
up_down<='0' ; --count down
wait for (14*CLOCK_PERIOD);

wait ;

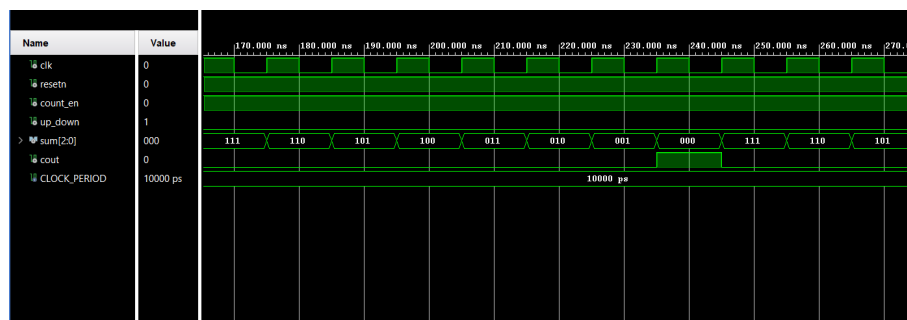
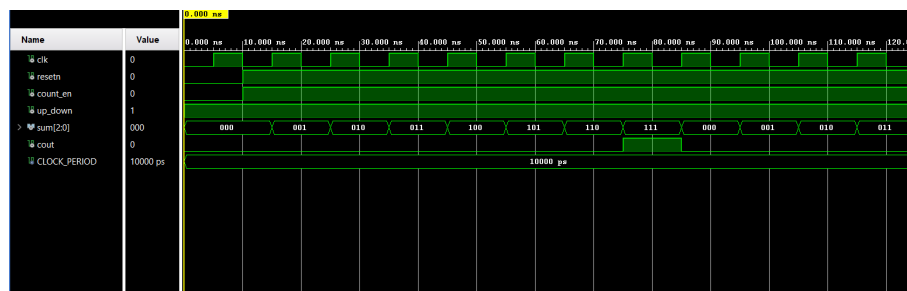
end process;

GEN_CLK : process
begin
    clk <= '0';
    wait for (CLOCK_PERIOD / 2);
    clk <= '1';
    wait for (CLOCK_PERIOD / 2);
end process;
end tb;

```

Two simulation snapshots are presented below in order to illustrate both upward and downward counting.

As shown in the simulation results, an “overflow indication” is generated by setting `cout` to 1 both when the counter reaches 7 while counting upward and when it reaches 0 while counting downward.



The above simulation results demonstrate the correct operation of the counter in both the upward-counting and downward-counting modes.

2. Up Counter with Limit

The VHDL code used to describe the limited up counter is presented below:

```

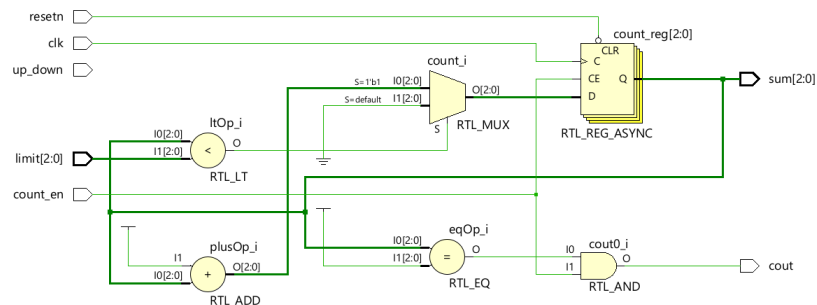
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity counter_up_down_lim is
    port(
        clk,resetn,count_en,up_down : in std_logic;
        limit : in std_logic_vector(2 downto 0);
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end counter_up_down_lim;

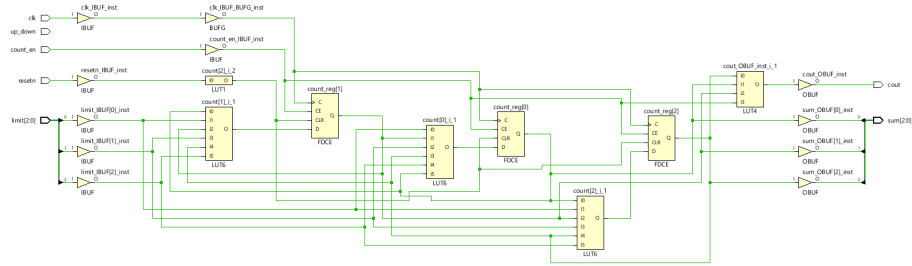
architecture rtl_lim of counter_up_down_lim is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, resetn, limit)
    begin
        if resetn='0' then
            -- Asynchronous reset
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en = '1' then
                -- Count only when count_en='1'
                if count<limit then
                    count <= count+1;
                else
                    -- Otherwise, reset the counter
                    count<=(others=>'0');
                end if;
            end if;
        end if;
    end process;
    sum<= count;
    cout <= '1' when count=7 and count_en='1' else '0';
end rtl_lim;

```

The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



The following testbench was implemented for the simulation:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity count_up_limit_tb is
-- Port ( );
end count_up_limit_tb;

architecture tb of count_up_limit_tb is

-- COMPONENT
component counter_up_down_lim is
    port(
        clk, resetn, count_en, up_down : in std_logic;
        limit : in std_logic_vector(2 downto 0);
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end component;

-- SIGNALS
signal clk : std_logic := '0';
signal resetn : std_logic := '0';
signal count_en : std_logic := '0';
signal up_down : std_logic := '0';
signal limit : std_logic_vector(2 downto 0) := "000";
signal sum : std_logic_vector(2 downto 0);
signal cout : std_logic;

-- CONSTANTS
constant CLOCK_PERIOD : time := 10 ns;

begin

DUT : counter_up_down_lim
port map(
    clk => clk,
    resetn => resetn,
    count_en => count_en,
    up_down => up_down,
    limit => limit,
    sum => sum,
    cout => cout
);

STIMULUS : process
begin
    resetn <= '1';

```

```

        wait for (24*CLOCK_PERIOD);
        resetn <= '0';
        wait for (CLOCK_PERIOD);
    end process;

COUNT : process
begin
    count_en <= '1';
    wait for (15*CLOCK_PERIOD);
    count_en <= '0';
    wait for (4*CLOCK_PERIOD);
end process;

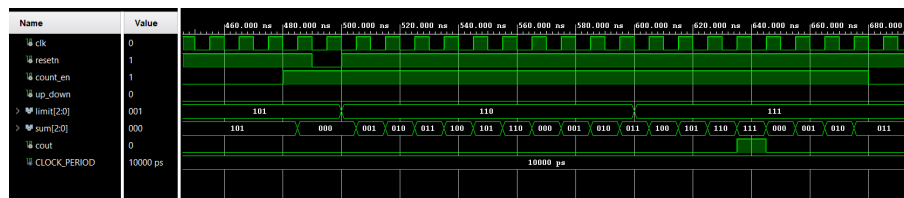
GEN_LIMIT : process
begin
    limit <= (limit) + 1;
    wait for (8*CLOCK_PERIOD);
end process;

GEN_CLK : process
begin
    clk <= '0';
    wait for (CLOCK_PERIOD / 2);
    clk <= '1';
    wait for (CLOCK_PERIOD / 2);
end process;

end tb;

```

An indicative portion of the simulation results demonstrating the correct operation of the code is shown below:



It can be observed that the counter is limited at each moment by the upper limit currently applied to the circuit.

Exercise 1: Half Adder using Dataflow Description

The VHDL code used to describe the half adder using a dataflow architecture is presented below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
    Port (
        i_a,i_b: in std_logic;

        o_sum ,o_carry : out std_logic
    );
end half_adder;

architecture Dataflow of half_adder is

```

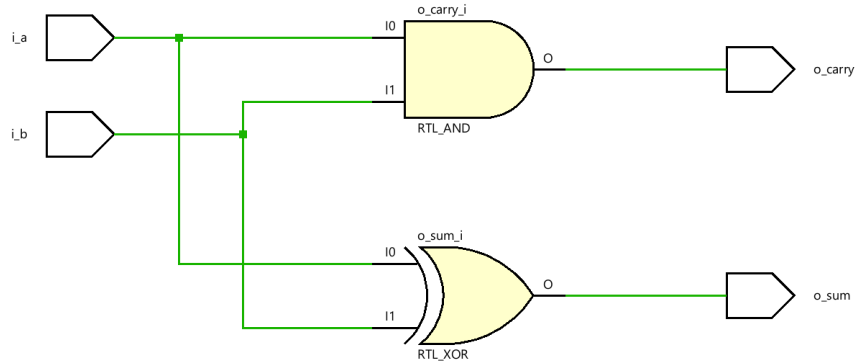
```

begin
    o_sum  <= i_a xor i_b;
    o_carry <= i_a and i_b;

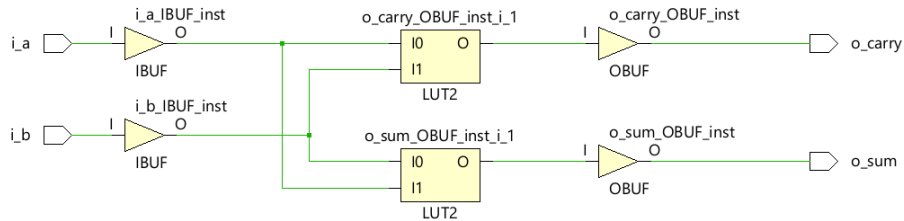
end Dataflow;

```

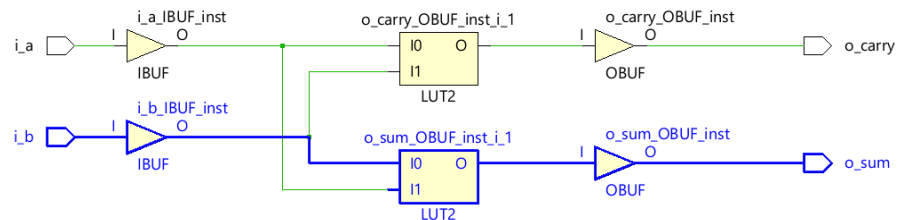
The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



Using the timing summary of the circuit, the critical path can be identified as shown below:



As shown below, the propagation delay of the circuit's critical path is 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	i_b	o_sum	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	i_b	o_carry	5.351	3.752	1.599	∞	input port clock		

Identifying the critical path is important because it allows us to determine which data path introduces the largest delay in the design. Therefore, if timing improvements are required, we know which path should be optimized. Furthermore, the maximum propagation delay directly affects the maximum operating frequency of the circuit.

The following testbench is used to simulate and verify the correct operation of the circuit:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder_tb is
-- Port ( );
end half_adder_tb;

architecture tb of half_adder_tb is

-----component-----
component half_adder is
  Port (
    i_a, i_b: in std_logic;

    o_sum ,o_carry : out std_logic
  );
end component;

-----signals-----
signal i_a,i_b,o_sum ,o_carry : std_logic := '0' ;

-----constant-----
constant CLOCK_PERIOD : time := 10 ns;

begin

  DUT: half_adder
  port map (
    i_a=>i_a,
    i_b=>i_b,

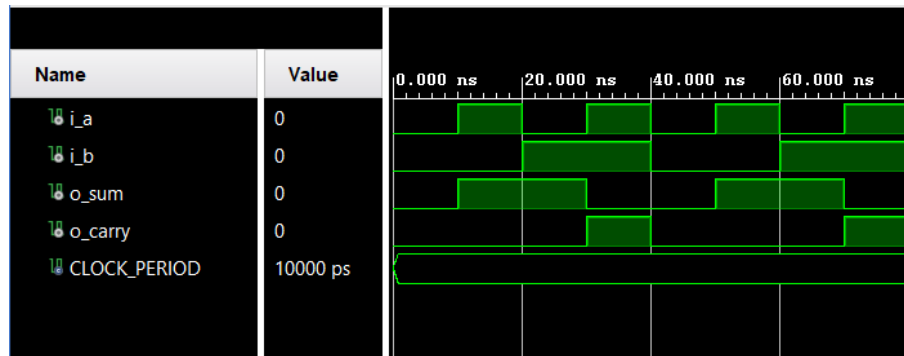
    o_sum=>o_sum,
    o_carry=>o_carry
  );

  GEN_IA : process
  begin
    i_a <= '0';
    wait for (CLOCK_PERIOD );
    i_a <= '1';
    wait for (CLOCK_PERIOD );
  end process;

  GEN_IB : process
  begin
    i_b <= '0';
    wait for (2*CLOCK_PERIOD );
    i_b <= '1';
    wait for (2*CLOCK_PERIOD );
  end process;

end tb;

```



The simulation results confirm the correct operation of the circuit for all possible input combinations.

Exercise 2: Full Adder using Structural Description

The VHDL code used to describe the full adder using a structural architecture is presented below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port (
        i_a, i_b, i_cin : in std_logic;
        o_sum, o_carry : out std_logic
    );
end full_adder;

architecture structural of full_adder is

    component half_adder is
        Port (
            i_a, i_b : in std_logic;
            o_sum, o_carry : out std_logic
        );
    end component;

    signal o_sum_mid : std_logic;
    signal o_carry_mid : std_logic;
    signal o_c : std_logic;

begin

    HA1 : half_adder
        port map(
            i_a => i_a,
            i_b => i_b,
            o_sum => o_sum_mid,
            o_carry => o_carry_mid
        );

    HA2 : half_adder
        port map(
            i_a => o_sum_mid,
            i_b => i_cin,
            o_sum => o_sum,
            o_carry => o_c
        );
end structural;
```

```

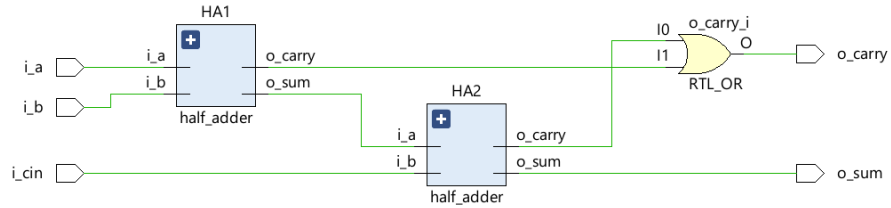
    );

    o_carry <= o_c or o_carry_mid;

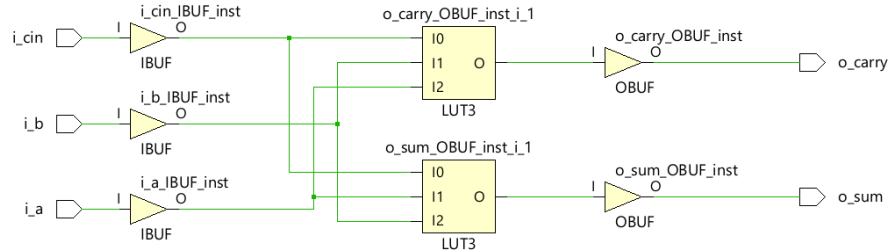
end structural;

```

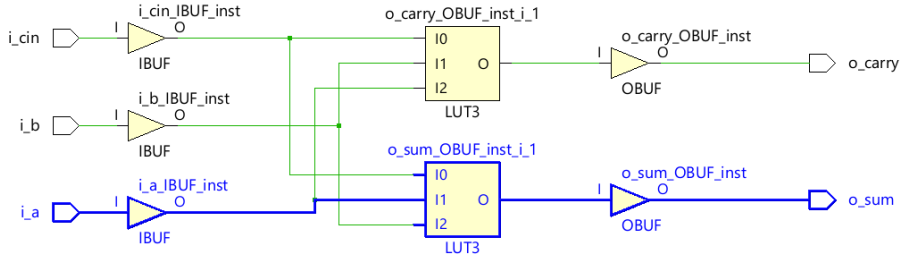
The following schematic is obtained using the Elaborate Design option:



As expected, the elaborated design shows that the full adder consists of two half adders and one OR gate. The following schematic is obtained after synthesis:



Using the timing summary of the circuit, the critical path can be identified as shown below:



As shown below, the propagation delay of the circuit's critical path is 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	i_a	o_sum	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	i_a	o_carry	5.351	3.752	1.599	∞	input port clock		

The following testbench is used to simulate and verify the correct operation of the circuit:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_tb is

```

```

-- Port ( );
end full_adder_tb;

architecture tb of full_adder_tb is

    component full_adder is
        Port (
            i_a, i_b, i_cin      : in std_logic;
            o_sum, o_carry : out std_logic
        );
    end component;

    signal i_a , i_b , i_cin , o_sum , o_carry : std_logic := '0' ;

    constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT : full_adder
        port map
        (
            i_a=>i_a,
            i_b=>i_b,
            i_cin=>i_cin,
            o_sum=>o_sum,
            o_carry=>o_carry
        );

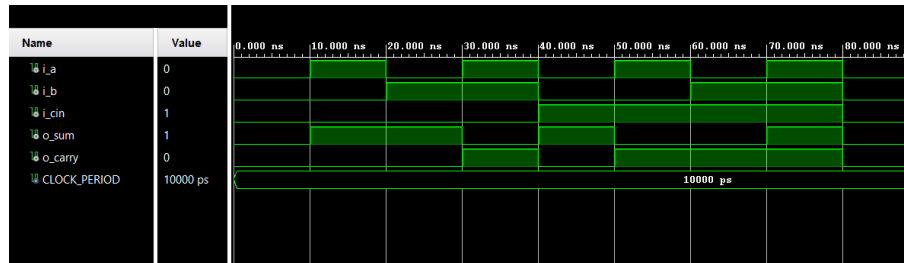
    IN_A : process
    begin
        i_a <= '0';
        wait for (CLOCK_PERIOD );
        i_a <= '1';
        wait for (CLOCK_PERIOD );
    end process;

    IN_B : process
    begin
        i_b <= '0';
        wait for (2*CLOCK_PERIOD );
        i_b <= '1';
        wait for (2*CLOCK_PERIOD );
    end process;

    IN_carry : process
    begin
        i_cin <= '0';
        wait for (4*CLOCK_PERIOD );
        i_cin <= '1';
        wait for (4*CLOCK_PERIOD );
    end process;

end tb;

```



The simulation results confirm the correct operation of the circuit for all possible input combinations.

Exercise 3: 4-bit Parallel Adder using Structural Description

The VHDL code used to describe the 4-bit parallel adder using a structural architecture is presented below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA_4bit is
    Port (
        i_a ,i_b :in std_logic_vector (3 downto 0);
        i_cin : in std_logic;
        o_sum :out std_logic_vector (3 downto 0);
        o_cout : out std_logic
    );
end FA_4bit;

architecture structural of FA_4bit is

    component full_adder is
        Port (
            i_a, i_b,i_cin      : in std_logic;
            o_sum, o_carry : out std_logic
        );
    end component;

    signal carry01,carry12,carry23 : std_logic := '0';

begin

    FA1 : full_adder
        port map (
            i_a    => i_a(0),
            i_b    => i_b(0),
            i_cin  => i_cin,
            o_sum  => o_sum(0),
            o_carry => carry01
        );

    FA2 : full_adder
        port map (
            i_a    => i_a(1),
            i_b    => i_b(1),
            i_cin  => carry01,
            o_sum  => o_sum(1),
            o_carry => carry12
        );

    FA3 : full_adder
        port map (
            i_a    => i_a(2),
            i_b    => i_b(2),
            i_cin  => carry12,
            o_sum  => o_sum(2),
            o_carry => carry23
        );

    FA4 : full_adder
        port map (
            i_a    => i_a(3),
            i_b    => i_b(3),
            i_cin  => carry23,
            o_sum  => o_sum(3),
            o_carry => o_cout
        );

end structural;
```

```

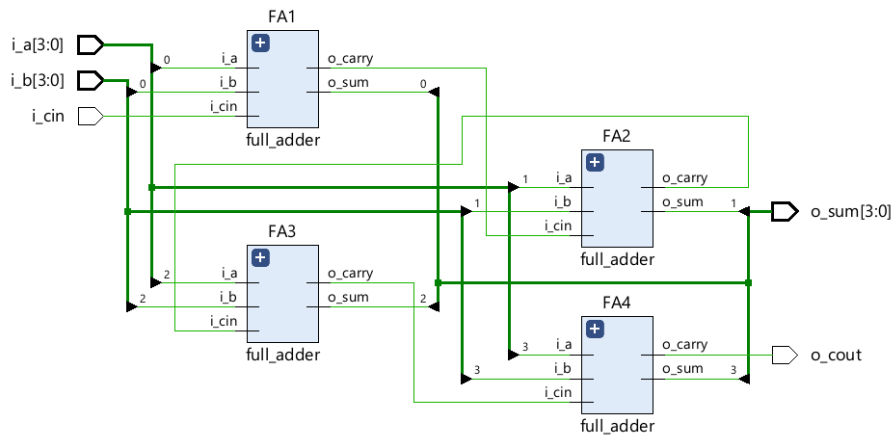
FA3 : full_adder
port map (
    i_a    => i_a(2),
    i_b    => i_b(2),
    i_cin  => carry12,
    o_sum  => o_sum(2),
    o_carry => carry23
);

FA4 : full_adder
port map (
    i_a    => i_a(3),
    i_b    => i_b(3),
    i_cin  => carry23,
    o_sum  => o_sum(3),
    o_carry => o_cout
);

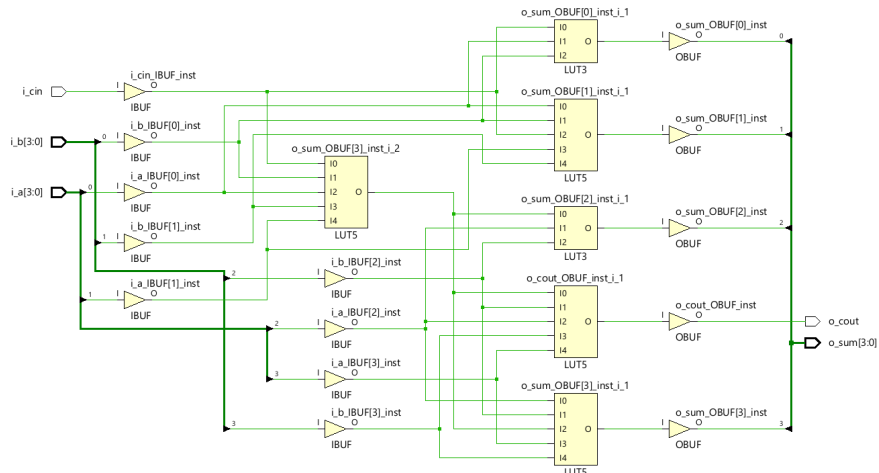
end structural;

```

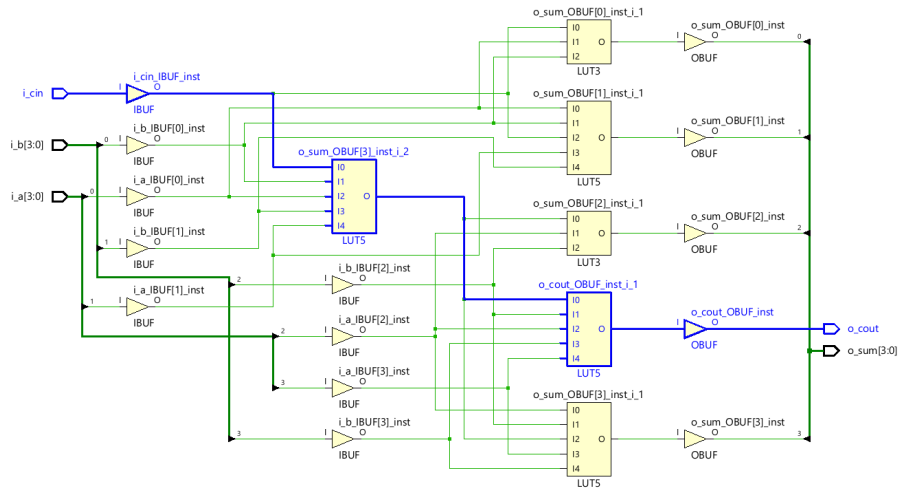
The following schematic is obtained using the Elaborate Design option:



As expected, the elaborated design shows that the 4-bit parallel adder consists of four full adders. The following schematic is obtained after synthesis:



Using the timing summary of the circuit, the critical path can be identified as shown below:



As shown below, the propagation delay of the circuit's critical path is 5.970 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exce
Path 1	∞	4	5	3	i_cin	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 2	∞	4	5	3	i_b[0]	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 3	∞	4	5	3	i_cin	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 4	∞	4	5	3	i_b[0]	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 5	∞	4	5	3	i_cin	o_sum[2]	5.970	3.904	2.066	∞	input port clock		
Path 6	∞	4	5	3	i_b[0]	o_sum[2]	5.970	3.904	2.066	∞	input port clock		
Path 7	∞	4	5	3	i_cin	o_sum[2]	5.970	3.904	2.066	∞	input port clock		

The following testbench is used to simulate and verify the correct operation of the circuit:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity FA_4bit_tb is
-- Port ( );
end FA_4bit_tb;

architecture tb of FA_4bit_tb is

component FA_4bit is
Port (
    i_a ,i_b :in std_logic_vector (3 downto 0);
    i_cin : in std_logic;
    o_sum :out std_logic_vector (3 downto 0);
    o_cout : out std_logic
);
end component;

signal i_cin ,o_cout :std_logic := '0';
signal i_a, i_b , o_sum :std_logic_vector(3 downto 0) := "0000";

constant CLOCK_PERIOD : time := 10 ns;

begin
    DUT : FA_4bit
    port map(
        i_a=>i_a,

```

```

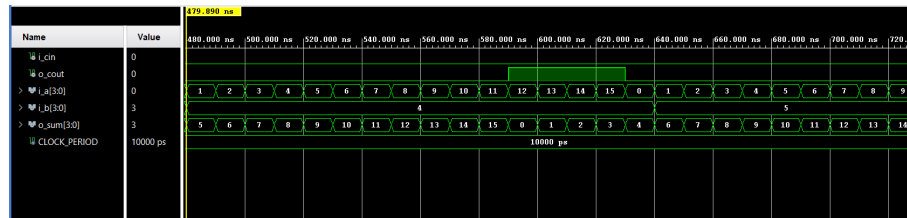
        i_b=>i_b,
        o_sum=>o_sum,
        i_cin=>i_cin,
        o_cout=>o_cout
    );

    IN_A : process
    begin
        i_a <= i_a+1;
        wait for (CLOCK_PERIOD );
    end process;

    IN_B : process
    begin
        i_b <= i_b+1;
        wait for (16*CLOCK_PERIOD);
    end process;

end tb;

```



The simulation results confirm the correct operation of the circuit for all tested input combinations. It should be noted that not all possible combinations are included, since their total number is very large. Instead, representative combinations were selected to verify the correct operation of the circuit.

Exercise 3: BCD Full Adder using Structural Description

In Binary-Coded Decimal (BCD) representation, each decimal digit of a number is represented separately using four binary bits. For example, the decimal number 12 is represented in BCD as 0001 0010, where 0001 corresponds to the digit 1 and 0010 corresponds to the digit 2.

For the BCD adder, when the result of the addition is less than or equal to 9, the carry output is 0. When the result is greater than 9, the carry output of the first adder becomes 1. This signal is then used to add the value 6 to the intermediate sum produced by the first adder, ensuring that the final result is a valid BCD representation. The VHDL code used to describe the BCD full adder using a structural architecture is presented below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BDC_adder is
    Port (
        i_a ,i_b :in std_logic_vector (3 downto 0);
        i_cin : in std_logic;
        o_bcd0 :out std_logic_vector (3 downto 0);
        o_bcd1: out std_logic
    );
end BDC_adder;

architecture Structural of BDC_adder is

```

```

component FA_4bit is
  Port (
    i_a ,i_b :in std_logic_vector (3 downto 0);
    i_cin : in std_logic;
    o_sum :out std_logic_vector (3 downto 0);
    o_cout : out std_logic
  );
end component;

signal o_cout :std_logic := '0';
signal o_sum :std_logic_vector(3 downto 0) := "0000";
signal checker: std_logic :='0';
begin
  FA1 : FA_4bit
  port map(
    i_a=>i_a,
    i_b=>i_b,
    i_cin=>i_cin,
    o_sum=>o_sum,
    o_cout=>o_cout
  );

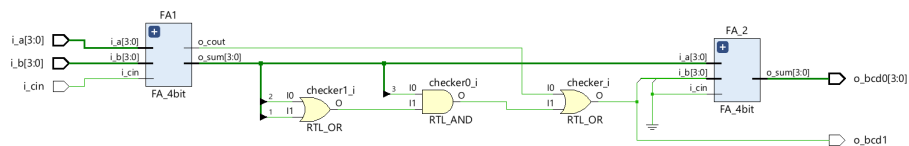
  checker<=o_cout or ((o_sum(3)) and (o_sum(2) or o_sum(1)));
  o_bcd1<=checker;

  FA_2: FA_4bit
  port map(
    i_a=>o_sum,
    i_b=>'0'&checker&checker&'0',
    i_cin=>'0',
    o_sum=>o_bcd0
  );

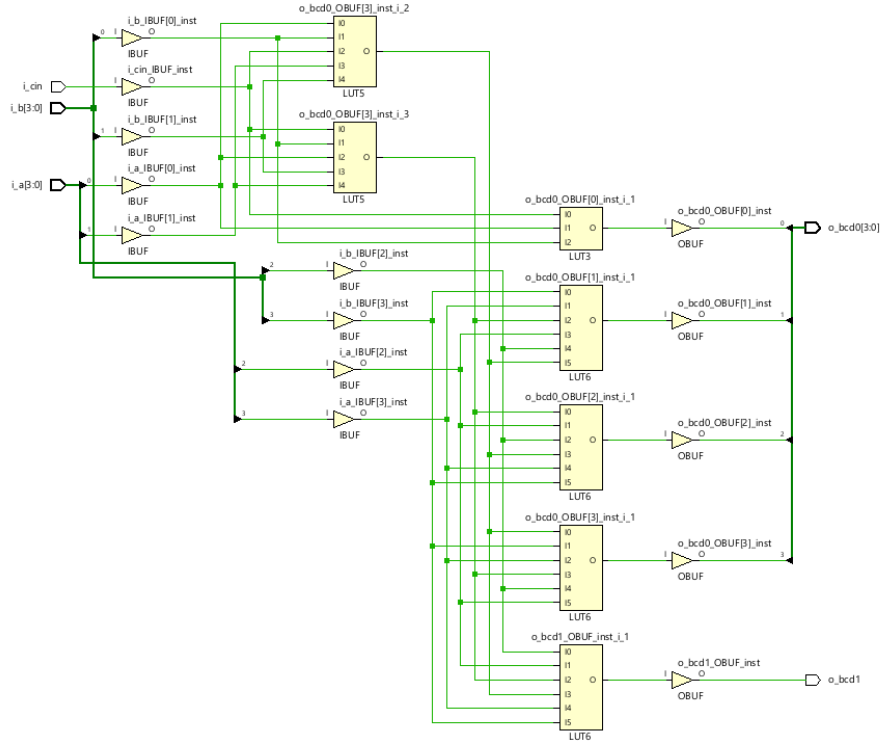
end Structural;

```

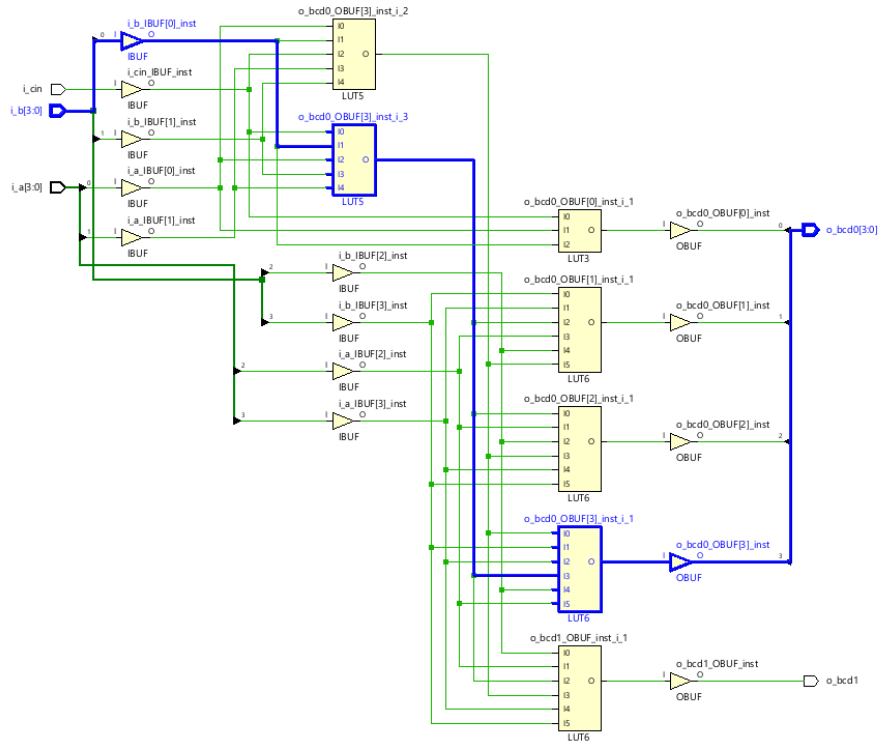
The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



Using the timing summary of the circuit, the critical path can be identified as shown below:



As shown below, the propagation delay of the circuit's critical path is 5.976 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Excepti
Path 1	∞	4	5	4	i_b[0]	o_bcd0[3]	5.976	3.904	2.072	∞	input port clock		
Path 2	∞	4	5	4	i_b[1]	o_bcd0[1]	5.948	3.876	2.072	∞	input port clock		
Path 3	∞	4	5	4	i_b[1]	o_bcd0[2]	5.948	3.876	2.072	∞	input port clock		

The following testbench is used to simulate and verify the correct operation of the circuit:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity bcd_adder_tb is
-- Port ( );
end bcd_adder_tb;

architecture tb of bcd_adder_tb is

component BDC_adder is
Port (
    i_a ,i_b :in std_logic_vector (3 downto 0);
    i_cin : in std_logic;
    o_bcd0 :out std_logic_vector (3 downto 0);
    o_bcd1: out std_logic
);
end component;

signal i_a,i_b,o_bcd0 : std_logic_vector (3 downto 0) := "0000";
signal i_cin ,o_bcd1 : std_logic := '0';

constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT: BDC_adder
    port map (

        i_a=>i_a,
        i_b=>i_b,
        i_cin=>i_cin,
        o_bcd0=>o_bcd0,
        o_bcd1=>o_bcd1

    );

    IN_A : process
    begin
        if i_a = "1010" then
            i_a <= "0000";
        else
            i_a <= i_a + 1;
        end if;
        wait for CLOCK_PERIOD;
    end process;

    IN_B : process
    begin
        if i_b = "1010" then
            i_b <= "0000";
        else
            i_b <= i_b + 1;
        end if;
        wait for 10 * CLOCK_PERIOD;
    end process;

```

```

end process;

end tb;

```



The simulation results confirm the correct operation of the circuit for all tested input combinations. It should be noted that not all possible combinations are included, since their total number is very large. Instead, representative combinations were selected to verify the correct operation of the circuit.

Exercise 4: 4-Digit BCD Parallel Adder using Structural Description

The VHDL code used to describe the 4-digit BCD parallel adder using a structural architecture is presented below:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BCD_4digits is
    Port(
        i_a0, i_a1, i_a2, i_a3 : in std_logic_vector(3 downto 0);
        i_b0, i_b1, i_b2, i_b3 : in std_logic_vector(3 downto 0);
        o_bcd0, o_bcd1, o_bcd2, o_bcd3 : out std_logic_vector(3 downto 0);
        o_bcd_final : out std_logic
    );
end BCD_4digits;

architecture structural of BCD_4digits is

    component BDC_adder is
        Port (
            i_a, i_b : in std_logic_vector(3 downto 0);
            i_cin : in std_logic;
            o_bcd0 : out std_logic_vector(3 downto 0);
            o_bcd1 : out std_logic
        );
    end component;

    signal bcd01, bcd12, bcd23 : std_logic := '0';

begin

    BCD0 : BDC_adder
        port map(
            i_a => i_a0,
            i_b => i_b0,
            i_cin => '0',

```

```

        o_bcd0 => o_bcd0,
        o_bcd1 => bcd01
    );

BCD1 : BDC_adder
port map(
    i_a    => i_a1,
    i_b    => i_b1,
    i_cin  => bcd01,
    o_bcd0 => o_bcd1,
    o_bcd1 => bcd12
);

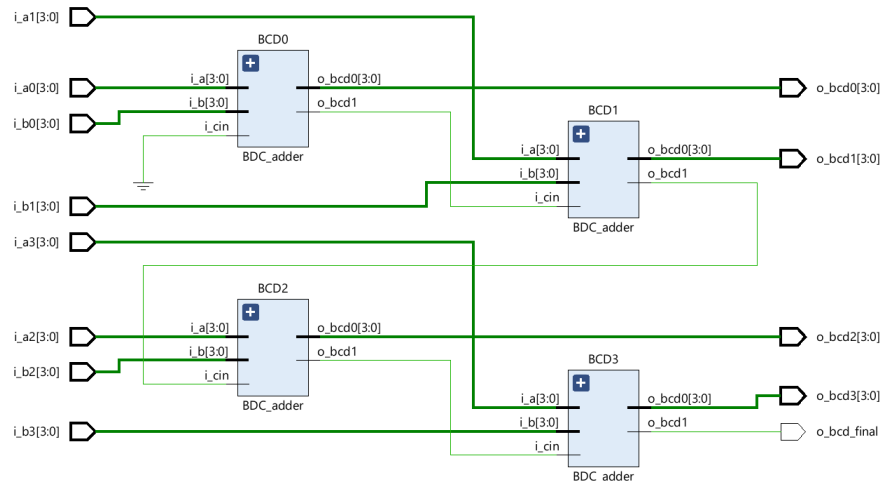
BCD2 : BDC_adder
port map(
    i_a    => i_a2,
    i_b    => i_b2,
    i_cin  => bcd12,
    o_bcd0 => o_bcd2,
    o_bcd1 => bcd23
);

BCD3 : BDC_adder
port map(
    i_a    => i_a3,
    i_b    => i_b3,
    i_cin  => bcd23,
    o_bcd0 => o_bcd3,
    o_bcd1 => o_bcd_final
);

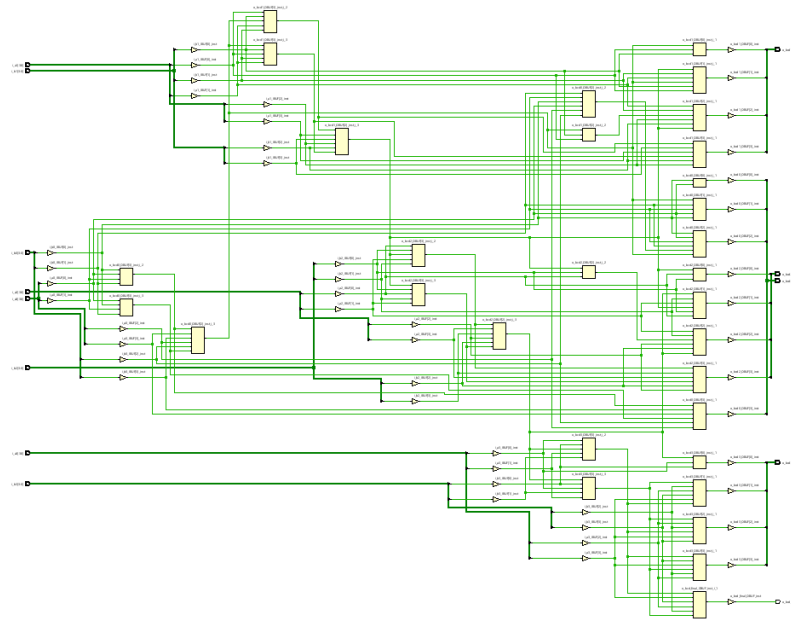
end structural;

```

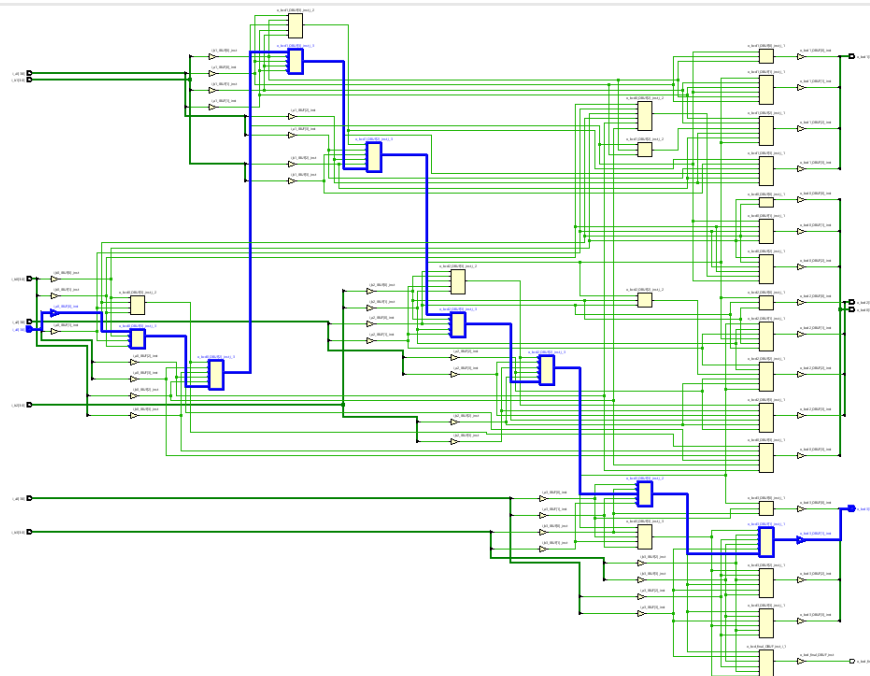
The following schematic is obtained using the Elaborate Design option:



The following schematic is obtained after synthesis:



Using the timing summary of the circuit, the critical path can be identified as shown below:



As shown below, the propagation delay of the circuit's critical path is 9.533 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 1	∞	10	11	7	i_a0[0]	o_bcd3[1]	9.533	4.636	4.897	∞	input port clock	
Path 2	∞	10	11	7	i_a0[0]	o_bcd3[2]	9.533	4.636	4.897	∞	input port clock	
Path 3	∞	10	11	7	i_a0[0]	o_bcd3[3]	9.527	4.630	4.897	∞	input port clock	
Path 4	∞	10	11	7	i_a0[0]	o_bcd_final	9.527	4.630	4.897	∞	input port clock	
Path 5	∞	9	10	7	i_a0[0]	o_bcd2[1]	8.936	4.512	4.424	∞	input port clock	

The following testbench is used to simulate and verify the correct operation of the circuit:

```
library IEEE;
```

```

use IEEE.STD_LOGIC_1164.ALL;

entity BCD_4dig_tb is
-- Port ( );
end BCD_4dig_tb;

architecture tb of BCD_4dig_tb is

component BCD_4digits is
    Port(
        i_a0, i_a1, i_a2, i_a3 : in std_logic_vector(3 downto 0);
        i_b0, i_b1, i_b2, i_b3 : in std_logic_vector(3 downto 0);
        o_bcd0, o_bcd1, o_bcd2, o_bcd3 : out std_logic_vector(3 downto 0);
        o_bcd_final : out std_logic
    );
end component;

signal i_a0, i_a1, i_a2, i_a3 : std_logic_vector(3 downto 0):=(others => '0');
signal i_b0, i_b1, i_b2, i_b3 : std_logic_vector(3 downto 0):=(others => '0');
signal o_bcd0,o_bcd1,o_bcd2,o_bcd3: std_logic_vector(3 downto 0):= (others => '0');
signal o_bcd_final :std_logic:='0';
constant CLOCK_PERIOD : time := 10 ns;

begin

DUT : BCD_4digits
    port map(
        i_a0      => i_a0,
        i_a1      => i_a1,
        i_a2      => i_a2,
        i_a3      => i_a3,
        i_b0      => i_b0,
        i_b1      => i_b1,
        i_b2      => i_b2,
        i_b3      => i_b3,
        o_bcd0     => o_bcd0,
        o_bcd1     => o_bcd1,
        o_bcd2     => o_bcd2,
        o_bcd3     => o_bcd3,
        o_bcd_final => o_bcd_final
    );

stimulus : process
begin

    -- TEST 1: 0000 + 0000 = 0000
    i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0000";
    i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0000";
    wait for 2*CLOCK_PERIOD ;

    -- TEST 2: 0001 + 0002 = 0003
    i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0001";
    i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0010";
    wait for 2*CLOCK_PERIOD ;

    -- TEST 3: 0005 + 0004 = 0009
    i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0101";
    i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0100";

```

```

wait for 2*CLOCK_PERIOD ;

-- TEST 4: 0005 + 0005 = 0010
i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0101";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0101";
wait for 2*CLOCK_PERIOD ;

-- TEST 5: 0009 + 0001 = 0010
i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "1001";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

-- TEST 6: 0099 + 0001 = 0100
i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "1001"; i_a0 <= "1001";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

-- TEST 7: 0999 + 0001 = 1000
i_a3 <= "0000"; i_a2 <= "1001"; i_a1 <= "1001"; i_a0 <= "1001";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

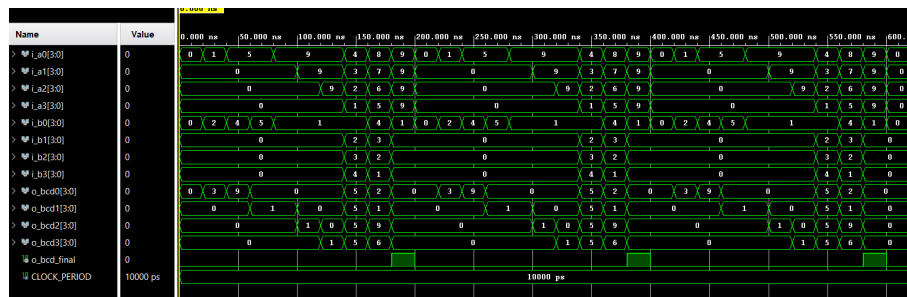
-- TEST 8: 1234 + 4321 = 5555
i_a3 <= "0001"; i_a2 <= "0010"; i_a1 <= "0011"; i_a0 <= "0100";
i_b3 <= "0100"; i_b2 <= "0011"; i_b1 <= "0010"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

-- TEST 9: 5678 + 1234 = 6912
i_a3 <= "0101"; i_a2 <= "0110"; i_a1 <= "0111"; i_a0 <= "1000";
i_b3 <= "0001"; i_b2 <= "0010"; i_b1 <= "0011"; i_b0 <= "0100";
wait for 2*CLOCK_PERIOD ;

-- TEST 10: 9999 + 0001 = 10000
i_a3 <= "1001"; i_a2 <= "1001"; i_a1 <= "1001"; i_a0 <= "1001";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

end process;
end tb;

```



The simulation results confirm the correct operation of the circuit for all tested input combinations. It should be noted that not all possible combinations are included, since their total number is very large. Instead, representative combinations were selected to verify the correct operation of the circuit.